

Análise de Código Java

BloomFilter — Estrutura de dados probabilística

■ Objetivo Geral

O código implementa um **Bloom Filter**, uma estrutura de dados probabilística usada para testar se um elemento *possivelmente pertence* a um conjunto, com custo de memória extremamente reduzido. Ele permite **falsos positivos**, mas nunca falsos negativos — ideal para cenários onde verificações rápidas são prioritárias, como caches, bancos de dados e sistemas de detecção de duplicatas.

■■ Lógica e Algoritmo

São utilizadas **três funções de hash polinomiais** com sementes distintas (7, 11, 31). Ao **adicionar** um elemento (add), calcula-se um índice para cada semente e marca-se o bit correspondente no BitSet. Para **consultar** (mightContain), os mesmos índices são recalculados — se qualquer bit estiver em 0, o elemento certamente não existe; se todos forem 1, ele *possivelmente* existe.

■ Conceitos Java Envolvidos

BitSet: estrutura nativa do Java para manipulação eficiente de bits; ocupa muito menos memória que arrays booleanos. **Arrays de primitivos** (int[] seeds) para iteração das sementes. **Math.abs** e operador % para garantir índices válidos. Uso de **enhanced for-loop** e método toCharArray() para iterar os caracteres da string durante o hashing.

■ Complexidade

Tanto a inserção quanto a consulta têm complexidade $O(k \cdot n)$, onde k é o número de funções de hash (constante = 3) e n é o comprimento da string — na prática, $O(n)$. O espaço ocupado é $O(m)$, com m sendo o tamanho do BitSet (1.000 bits $\approx$ 125 bytes), independentemente do número de elementos inseridos.

■■ Decisões de Projeto e Limitações

As sementes foram escolhidas como números primos para reduzir colisões, mas a função de hash polinomial é simples e propensa a mais falsos positivos do que funções como **MurmurHash**. Não há suporte a **remoção de elementos** (limitação inerente ao Bloom Filter clássico). O tamanho fixo do BitSet (1.000 bits) não se ajusta dinamicamente ao volume de dados, o que pode aumentar a taxa de falsos positivos conforme mais elementos são inseridos.

■ Trecho-Chave: Função de Hash

```
private int hash(String val, int seed) {  
    int h = 0;  
    for (char c : val.toCharArray())  
        h = h * seed + c; // hash polinomial  
    return Math.abs(h % size); // índice válido no BitSet  
}
```